

Interrupt Management



Registering an interrupt handler 1/2

The *managed* API is recommended:

```
int devm_request_irq(struct device *dev, unsigned int irq, irq_handler_t handler,
                     unsigned long irq_flags, const char *devname, void *dev_id);
```

- `device` for automatic freeing at device or module release time.
- `irq` is the requested IRQ channel. For platform devices, use platform_get_irq() to retrieve the interrupt number.
- `handler` is a pointer to the IRQ handler function
- `irq_flags` are option masks (see next slide)
- `devname` is the registered name (for `/proc/interrupts`). For platform drivers, good idea to use `pdev->name` which allows to distinguish devices managed by the same driver (example: `44e0b000.i2c`).
- `dev_id` is an opaque pointer. It can typically be used to pass a pointer to a per-device data structure. It cannot be `NULL` as it is used as an identifier for freeing interrupts on a shared line.



Registering an interrupt handler 2/2

Here are the most frequent `irq_flags` bit values in drivers (can be combined):

- IRQF_SHARED: interrupt channel can be shared by several devices.
  - When an interrupt is received, all the interrupt handlers registered on the same interrupt line are called.
  - This requires a hardware status register telling whether an IRQ was raised or not.
- IRQF_ONESHOT: for use by threaded interrupts (see next slides). Keeping the interrupt line disabled until the thread function has run.



Interrupt handler constraints

- No guarantee in which address space the system will be in when the interrupt occurs: can't transfer data to and from user space.
- Interrupt handler execution is managed by the CPU, not by the scheduler. Handlers can't run actions that may sleep, because there is nothing to resume their execution. In particular, need to allocate memory with GFP_ATOMIC.
- Interrupt handlers are run with all interrupts disabled on the local CPU (see https://lwn.net/Articles/380931). Therefore, they have to complete their job quickly enough, to avoiding blocking interrupts for too long.



/proc/interrupts on Raspberry Pi 2 (ARM, Linux 4.19)

```
           CPU0       CPU1       CPU2       CPU3
 17:    1005317          0          0          0  ARMCTRL-level   1 Edge      3f00b880.mailbox
 18:         36          0          0          0  ARMCTRL-level   2 Edge      VCHIQ doorbell
 40:          0          0          0          0  ARMCTRL-level  48 Edge      bcm2708_fb DMA
 42:     427715          0          0          0  ARMCTRL-level  50 Edge      DMA IRQ
 56:  478426356          0          0          0  ARMCTRL-level  64 Edge      dwc_otg, dwc_otg_pcd, dwc_otg_hcd:usb1
 80:     411468          0          0          0  ARMCTRL-level  88 Edge      mmc0
 81:        502          0          0          0  ARMCTRL-level  89 Edge      uart-pl011
161:          0          0          0          0  bcm2836-timer   0 Edge      arch_timer
162:   10963772    6378711   16583353    6406625  bcm2836-timer   1 Edge      arch_timer
165:          0          0          0          0  bcm2836-pmu     9 Edge      arm-pmu
FIQ:                                             usb_fiq
IPI0:         0          0          0          0  CPU wakeup interrupts
IPI1:         0          0          0          0  Timer broadcast interrupts
IPI2:   2625198    4404191    7634127    3993714  Rescheduling interrupts
IPI3:      3140      56405      49483      59648  Function call interrupts
IPI4:         0          0          0          0  CPU stop interrupts
IPI5:   2167923     477097    5350168     412699  IRQ work interrupts
IPI6:         0          0          0          0  completion interrupts Err:          0
```

Note: interrupt numbers shown on the left-most column are virtual numbers when the Device Tree is used. The physical interrupt numbers can be found in `/sys/kernel/debug/irq/irqs/<nr>` files when CONFIG_GENERIC_IRQ_DEBUGFS=y.



Interrupt handler prototype

- `irqreturn_t foo_interrupt(int irq, void *dev_id)`
  - `irq`, the IRQ number
  - `dev_id`, the per-device pointer that was passed to devm_request_irq()
- Return value
  - IRQ_HANDLED: recognized and handled interrupt
  - IRQ_NONE: used by the kernel to detect spurious interrupts, and disable the interrupt line if none of the interrupt handlers has handled the interrupt.
  - IRQ_WAKE_THREAD: handler requests to wake the handler thread (see next slides)



Typical interrupt handler's job

- Acknowledge the interrupt to the device (otherwise no more interrupts will be generated, or the interrupt will keep firing over and over again)
- Read/write data from/to the device
- Wake up any process waiting for such data, typically on a per-device wait queue:
  `wake_up_interruptible(&device_queue);`



Top half and bottom half processing

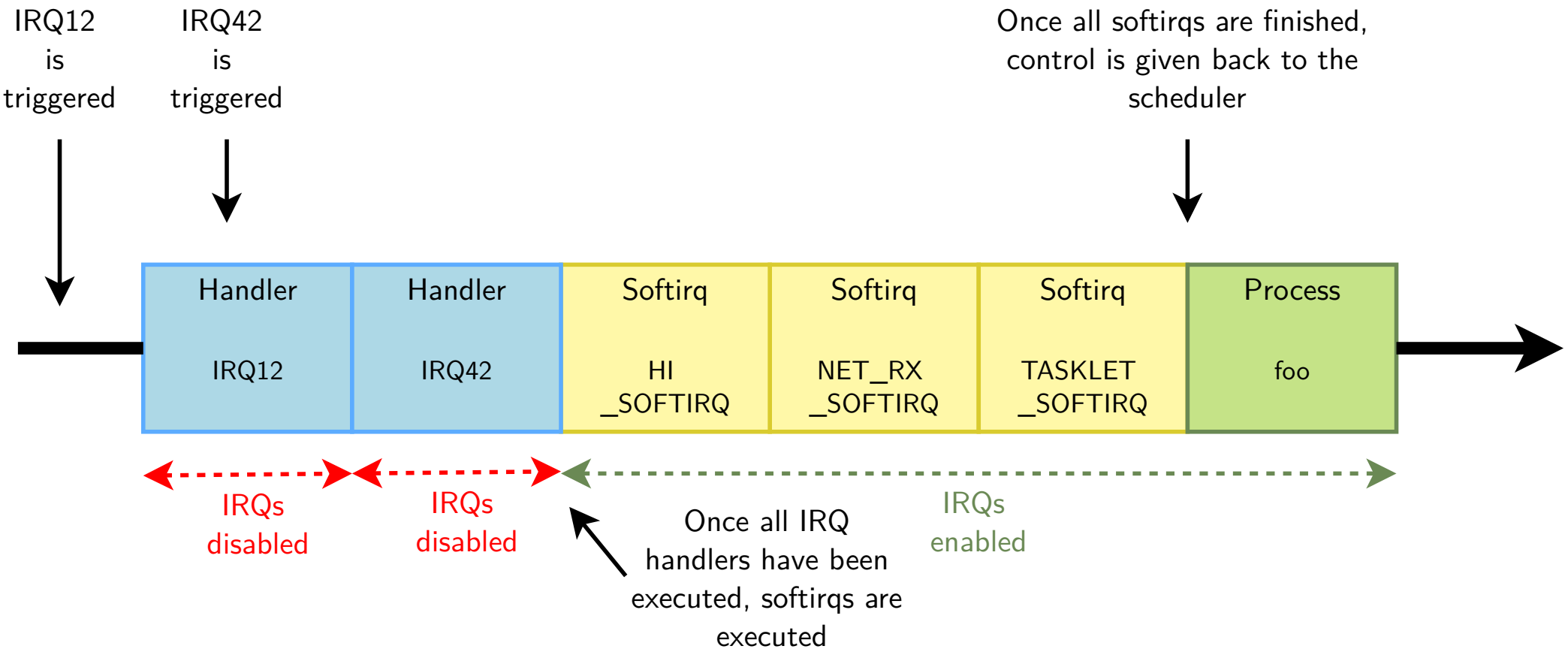
- Splitting the execution of interrupt handlers in 2 parts is sometimes needed
  - Hard IRQ handlers execute with all interrupts on the local CPU masked
    - No interrupt nesting mechanism, higher priority IRQs will be delayed
  - Need to block/sleep (eg. accessing I2C devices may involve sleeping)
- Top half
  - This is the real interrupt handler, which should complete as quickly as possible since all interrupts are disabled. It takes the data out of the device and if substantial post-processing is needed, schedules a bottom half to handle it.
- Bottom half
  - Is the general name for various mechanisms which allow to postpone the handling of interrupt-related work.
    - Implemented in Linux as softirqs, threaded handlers and workqueues
    - And yet, the abbreviation “bh” often means “softirqs” for historical reasons!



Softirqs

- Softirq handlers are callbacks executed once all interrupt handlers have completed, before the kernel resumes scheduling processes
  - They execute with all interrupts enabled
  - They run before the scheduler is in control, so sleeping is not allowed
  - A softirq handler can run simultaneously on multiple CPUs
- The number of softirqs is fixed, softirqs are not used by drivers, but by kernel subsystems (network, etc.). The list of softirqs is defined in include/linux/interrupt.h
- To avoid starving the system, there is typically a softirq budget (by default, softirqs can run 10 times in a row for a maximum time of 2ms) after which the callbacks are run in process context in a per CPU `ksoftirqd/N` kernel thread.
- A Tasklet is a deprecated mechanism which was dedicated to device drivers and was implemented on top of softirqs.

Softirq execution flow





Example usage of softirqs: NAPI

- Interface in the Linux kernel used for interrupt mitigation in network drivers
- Principle: when the network traffic exceeds a given threshold ("budget"), disable network interrupts and consume incoming packets through a polling function, instead of processing each new packet with an interrupt.
- This reduces overhead due to interrupts and yields better network throughput.
- The polling function is run by napi_schedule(), which uses NET_RX_SOFTIRQ.
- See https://en.wikipedia.org/wiki/New_API for details
- See also our commented network driver on https://bootlin.com/pub/drivers/r6040-network-driver-with-comments.c



Threaded interrupts

- It is possible to associate a threaded handler to a hard IRQ handler
  - The hard IRQ handler triggers it by returning IRQ_WAKE_THREAD
  - The threaded handler is executed inside a thread, in process context
- The interrupt line may be kept masked if the handler registration happened with IRQF_ONESHOT
- Sleeping/blocking is allowed in the threaded handler
  - The priority of the `irq/<nb>-<name>` thread can be tuned!
- Heavily used by `PREEMPT_RT`

```
int devm_request_threaded_irq(struct device *dev, unsigned int irq,
                              irq_handler_t handler, irq_handler_t thread_fn,
                              unsigned long flags, const char *name,
                              void *dev);
```

- `handler`: “hard IRQ” handler
- `thread_fn`: executed in a thread



Workqueues

- Workqueues are a general mechanism for deferring work. It is not limited in usage to handling interrupts.
  - Typically used for background jobs.
- Functions registered to run in workqueues are called works:
  - They can be created with the macro INIT_WORK()
  - When scheduled, they become threads (called workers) running in process context, which means:
    - All interrupts are enabled
    - Sleeping is allowed
  - Works can be queued on:
    - The default workqueue, with schedule_work()
    - A workqueue allocated by the subsystem or the drivers, with alloc_workqueue()
- The complete API is in include/linux/workqueue.h
- Example (drivers/crypto/atmel-i2c.c):

```
INIT_WORK(&work_data->work, atmel_i2c_work_handler);
schedule_work(&work_data->work);
```



Interrupt and deferred mechanisms execution constraints summary

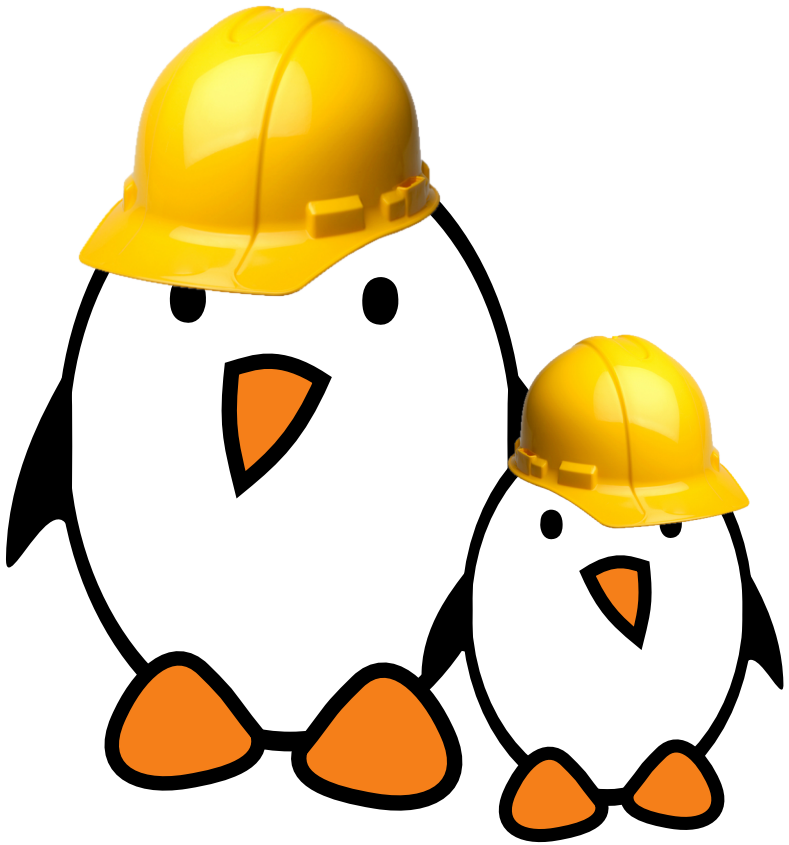
Mechanism	**Context**	**IRQs**	**Priority tuning**	**Can sleep**	**Typical Use**
Hard IRQ	Interrupt	Disabled (local CPU)	No, FIFO	No	Fast handling, gen. purpose
Softirq	Interrupt	Enabled	No, fixed priorities	No	Fast handling, net, timers, RCU
Softirq	Process (ksoftirqd)	Enabled	One kthread per CPU	No	Softirq overflow
Threaded IRQ	Process (irq thread)	Enabled (IRQF_ONESHOT)	Yes	Yes	Interrupt handling needing to block
Workqueue	Process (worker)	Enabled	Yes, per queue	Yes	General purpose background worker



Interrupt management summary

- Device driver
  - In the `probe()` function, for each device, use devm_request_irq() to register an interrupt handler for the device's interrupt channel.
- Interrupt handler
  - Called when an interrupt is raised.
  - Acknowledge the interrupt
  - If needed, trigger a bottom half mechanism (typically taking care of handling data)
  - Wake up processes waiting for the data on a per-device queue
- Device driver
  - In the `remove()` function, for each device, the interrupt handler is automatically unregistered.

Practical lab - Interrupts



- Adding read capability to the character driver developed earlier.
- Register an interrupt handler for each device.
- Waiting for data to be available in the read file operation.
- Waking up the code when data are available from the devices.